CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering
Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)
Practical File – ODD Semester 2026–27

Student ID: 24DIT074

Student Name: NANDAN VAKANI

PRACTICAL 2

1. Problem Definition: Programmatically interface with diverse file-based, API-driven, and relational source database environments. Students will write generation engines to simulate transactional workloads, profile the incoming files to automatically discover schemas, isolate data quality violations (such as null entry leaks and format divergence), and establish appropriate operational ingestion criteria.

Dataset/Test Data: Programmatically simulated customer profile tables, semi-structured API transaction logs containing nested objects, and unstructured configuration text files.

Tools/Technology To Be Used: Python 3.x, Faker library, Pandas, Requests HTTP library, SQLite/PostgreSQL, and Git.

2. Solution :

The practical was implemented using Python 3.x with the following workflow:

- ➤ Step 1 – Generate Synthetic Data (generate_data.py)
- ➤ Step 2 – Store Data in SQLite Database (database.py)
- ➤ Step 3 – Read Data from SQLite (read_database.py)
- ➤ Step 4 – Fetch Data from Public REST API (api_data.py)
- ➤ Step 5 – Profile Dataset (profile.py)
- ➤ Step 6 – Validate Email Addresses (validate_email.py)
- ➤ Step 7 – Detect Duplicate Records (duplicate.py)
- ➤ Step 8 – Clean Dataset (clean_data.py)

2.1 generate_data.py – Synthetic Data Generation

The Faker library is used to produce 100 realistic customer records and export them to customers.csv. Each record contains a unique CustomerID, Name, Email, Age, and City.



2.2 database.py – SQLite Storage

Loads the CSV data into an SQLite database (customers.db) using Pandas to_sql(), replacing the table if it already exists.

database.py

Practical_2 > database.py > ...

```python
import sqlite3
import pandas as pd

conn = sqlite3.connect("customers.db")

df = pd.read_csv("customers.csv")

df.to_sql("Customers",conn,if_exists="replace",index=False)

print("Database Created")

conn.close()
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\database.py"
Database Created

[Done] exited with code=0 in 1.145 seconds
```

2.3 read_database.py – Reading from SQLite

Executes a SELECT * query against the Customers table and loads results into a Pandas DataFrame for display and downstream processing.

read_database.py

Practical_2 > read_database.py > ...

```python
import sqlite3
import pandas as pd

# Connect to the database
conn = sqlite3.connect("customers.db")

# Read data from the Customers table
df = pd.read_sql("SELECT * FROM Customers", conn)

# Display the data
print(df)

# Close the connection
conn.close()
```

Problems Output Terminal Ports

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\read_database.py"
    CustomerID                Name  ... Age                City
0            1      Victor Mccarthy  ...  23  Port Amberborough
1            2         Kyle Serrano  ...  35   West Darlenebury
2            3        Steven Miller  ...  45   Lake Georgemouth
3            4    Christopher Davis  ...  31   North Josephtown
4            5         Jamie Murphy  ...  49          Ninafurt
..         ...                  ...  ...  ..                ...
95          96         Vicki Harris  ...  38          Loriton
96          97         Joshua Rubio  ...  63  New Jacobborough
97          98  Richard Gregory MD   ...  22      East Brandon
98          99         Caleb Murphy  ...  42      Clarencebury
99         100        Brenda Carson  ...  45          Tinaton

[100 rows x 5 columns]

[Done] exited with code=0 in 0.803 seconds
```

2.4 api_data.py – REST API Data Ingestion

Fetches a public JSON dataset from JSONPlaceholder to simulate third-party API data ingestion using the Requests library.

api_data.py

Practical_2 > api_data.py > ...

```python
import requests

url="https://jsonplaceholder.typicode.com/users"

response=requests.get(url)

print(response.json())
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\api_data.py"
[{'id': 1, 'name': 'Leanne Graham', 'username': 'Bret', 'email': 'Sincere@april.biz', 'address': {'street': 'Kulas Light', 'suite': 'Apt. 556', 'city': 'Gwenborough', 'zipcode': '92998-3874', 'geo': {'lat': '-37.3159', 'lng': '81.1496'}}, 'phone': '1-770-736-8031 x56442', 'website':
'hildegard.org', 'company': {'name': 'Romaguera-Crona', 'catchPhrase': 'Multi-layered client-server neural-net', 'bs': 'harness real-time e-markets'}}, {'id': 2, 'name': 'Ervin Howell', 'username': 'Antonette', 'email': 'Shanna@melissa.tv', 'address': {'street': 'Victor Plains', 'suite':
'Suite 879', 'city': 'Wisokyburgh', 'zipcode': '90566-7771', 'geo': {'lat': '-43.9509', 'lng': '-34.4618'}}, 'phone': '010-692-6593 x09125', 'website': 'anastasia.net', 'company': {'name': 'Deckow-Crist', 'catchPhrase': 'Proactive didactic contingency', 'bs': 'synergize scalable
supply-chains'}}, {'id': 3, 'name': 'Clementine Bauch', 'username': 'Samantha', 'email': 'Nathan@yesenia.net', 'address': {'street': 'Douglas Extension', 'suite': 'Suite 847', 'city': 'McKenziehaven', 'zipcode': '59590-4157', 'geo': {'lat': '-68.6102', 'lng': '-47.0653'}}, 'phone':
'1-463-123-4447', 'website': 'ramiro.info', 'company': {'name': 'Romaguera-Jacobson', 'catchPhrase': 'Face to face bifurcated interface', 'bs': 'e-enable strategic applications'}}, {'id': 4, 'name': 'Patricia Lebsack', 'username': 'Karianne', 'email': 'Julianne.OConner@kory.org', 'address':
{'street': 'Hoeger Mall', 'suite': 'Apt. 692', 'city': 'South Elvis', 'zipcode': '53919-4257', 'geo': {'lat': '29.4572', 'lng': '-164.2990'}}, 'phone': '493-170-9623 x156', 'website': 'kale.biz', 'company': {'name': 'Robel-Corkery', 'catchPhrase': 'Multi-tiered zero tolerance productivity',
'bs': 'transition cutting-edge web services'}}, {'id': 5, 'name': 'Chelsey Dietrich', 'username': 'Kamren', 'email': 'Lucio_Hettinger@annie.ca', 'address': {'street': 'Skiles Walks', 'suite': 'Suite 351', 'city': 'Roscoeview', 'zipcode': '33263', 'geo': {'lat': '-31.8129', 'lng': '62.5342'}},
'phone': '(254)954-1289', 'website': 'demarco.info', 'company': {'name': 'Keebler LLC', 'catchPhrase': 'User-centric fault-tolerant solution', 'bs': 'revolutionize end-to-end systems'}}, {'id': 6, 'name': 'Mrs. Dennis Schulist', 'username': 'Leopoldo_Corkery', 'email': 'Karley_Dach@jasper.
info', 'address': {'street': 'Norberto Crossing', 'suite': 'Apt. 950', 'city': 'South Christy', 'zipcode': '23505-1337', 'geo': {'lat': '-71.4197', 'lng': '71.7478'}}, 'phone': '1-477-935-8478 x6430', 'website': 'ola.org', 'company': {'name': 'Considine-Lockman', 'catchPhrase': 'Synchronised
bottom-line interface', 'bs': 'e-enable innovative applications'}}, {'id': 7, 'name': 'Kurtis Weissnat', 'username': 'Elwyn.Skiles', 'email': 'Telly.Hoeger@billy.biz', 'address': {'street': 'Rex Trail', 'suite': 'Suite 280', 'city': 'Howemouth', 'zipcode': '58804-1099', 'geo': {'lat': '24.
8918', 'lng': '21.8984'}}, 'phone': '210.067.6132', 'website': 'elvis.io', 'company': {'name': 'Johns Group', 'catchPhrase': 'Configurable multimedia task-force', 'bs': 'generate enterprise e-tailers'}}, {'id': 8, 'name': 'Nicholas Runolfsdottir V', 'username': 'Maxime_Nienow', 'email':
'Sherwood@rosamond.me', 'address': {'street': 'Ellsworth Summit', 'suite': 'Suite 729', 'city': 'Aliyaview', 'zipcode': '45169', 'geo': {'lat': '-14.3990', 'lng': '-120.7677'}}, 'phone': '586.493.6943 x140', 'website': 'jacynthe.com', 'company': {'name': 'Abernathy Group', 'catchPhrase':
'Implemented secondary concept', 'bs': 'e-enable extensible e-tailers'}}, {'id': 9, 'name': 'Glenna Reichert', 'username': 'Delphine', 'email': 'Chaim_McDermott@dana.io', 'address': {'street': 'Dayna Park', 'suite': 'Suite 449', 'city': 'Bartholomebury', 'zipcode': '76495-3109', 'geo':
{'lat': '24.6463', 'lng': '-168.8889'}}, 'phone': '(775)976-6794 x41206', 'website': 'conrad.com', 'company': {'name': 'Yost and Sons', 'catchPhrase': 'Switchable contextually-based project', 'bs': 'aggregate real-time technologies'}}, {'id': 10, 'name': 'Clementina DuBuque', 'username':
'Moriah.Stanton', 'email': 'Rey.Padberg@karina.biz', 'address': {'street': 'Kattie Turnpike', 'suite': 'Suite 198', 'city': 'Lebsackbury', 'zipcode': '31428-2261', 'geo': {'lat': '-38.2386', 'lng': '57.2232'}}, 'phone': '024-648-3804', 'website': 'ambrose.net', 'company': {'name': 'Hoeger
LLC', 'catchPhrase': 'Centralized empowering task-force', 'bs': 'target end-to-end models'}}]

[Done] exited with code=0 in 1.489 seconds
```

2.5 profile.py – Data Profiling

Produces a structural analysis of the dataset including column data types, descriptive statistics, and a null-value summary per column.

profile.py

Practical_2 > profile.py > ...

```python
import pandas as pd

df=pd.read_csv("customers.csv")

print(df.info())

print()

print(df.describe())

print()

print(df.isnull().sum())
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\profile.py"
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   CustomerID  100 non-null    int64
 1   Name        100 non-null    str
 2   Email       100 non-null    str
 3   Age         100 non-null    int64
 4   City        100 non-null    str
dtypes: int64(2), str(3)
memory usage: 4.0 KB
None

       CustomerID         Age
count  100.000000  100.000000
mean    50.500000   40.600000
std     29.011492   14.164260
min      1.000000   18.000000
25%     25.750000   29.000000
50%     50.500000   39.500000
75%     75.250000   52.500000
max    100.000000   65.000000

CustomerID    0
Name          0
Email         0
Age           0
City          0
dtype: int64

[Done] exited with code=0 in 0.746 seconds
```

2.6 validate_email.py – Email Format Validation

Applies a regular-expression pattern to every Email field and flags records whose address does not conform to the standard user@domain.ext format.

```python
from faker import Faker
import pandas as pd
import random

fake = Faker()

data = []

for i in range(100):
    data.append({
        "CustomerID": i + 1,
        "Name": fake.name(),
        "Email": fake.email(),
        "Age": random.randint(18, 65),
        "City": fake.city()
    })

df = pd.DataFrame(data)
df.to_csv("customers.csv", index=False)

print("customers.csv created successfully!")
print(df.head())
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\validate_email.py"
customers.csv created successfully!
   CustomerID             Name  ... Age            City
0           1  Michelle Nelson  ...  41      Baileyshire
1           2      Jacob Petty  ...  63    South Johnland
2           3      Kara Cannon  ...  30      Margaretton
3           4      Andrew Mann  ...  50    Andradechester
4           5      Glenda Hood  ...  32  Lake Rebeccaland

[5 rows x 5 columns]

[Done] exited with code=0 in 0.969 seconds
```

2.7 duplicate.py – Duplicate Record Detection

Scans the dataset for fully duplicate rows and reports the count. Since data is synthetically generated with unique CustomerIDs, no duplicates are expected.

```python
import pandas as pd

# Read the CSV file
df = pd.read_csv("customers.csv")

# Find duplicate rows
duplicates = df[df.duplicated()]

# Display duplicate records
if duplicates.empty:
    print("No duplicate records found.")
else:
    print("Duplicate Records:")
    print(duplicates)

# Count duplicate records
print("\nTotal Duplicate Records:", duplicates.shape[0])
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\duplicate.py"
No duplicate records found.

Total Duplicate Records: 0

[Done] exited with code=0 in 1.047 seconds
```

2.8 clean_data.py – Data Cleaning & Export

Removes all rows that contain at least one null value and exports the resulting clean dataset as clean_customers.csv, ready for ingestion into a downstream system.

clean_data.py

Practical_2 > clean_data.py > ...

```python
import pandas as pd

# Read the CSV file
df = pd.read_csv("customers.csv")

print("Before Cleaning:", len(df), "records")

# Remove rows with missing (null) values
clean_df = df.dropna()

print("After Cleaning:", len(clean_df), "records")

# Save the cleaned data to a new CSV file
clean_df.to_csv("clean_customers.csv", index=False)

print("\nCleaned data has been saved as 'clean_customers.csv'")
print("\nFirst 5 records of cleaned data:")
print(clean_df.head())
```

```
[Running] python -u "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_2\clean_data.py"
Before Cleaning: 100 records
After Cleaning: 100 records

Cleaned data has been saved as 'clean_customers.csv'

First 5 records of cleaned data:
   CustomerID            Name  ... Age             City
0           1  Michelle Nelson  ...  41       Baileyshire
1           2      Jacob Petty  ...  63    South Johnland
2           3      Kara Cannon  ...  30       Margaretton
3           4      Andrew Mann  ...  50    Andradechester
4           5      Glenda Hood  ...  32  Lake Rebeccaland

[5 rows x 5 columns]

[Done] exited with code=0 in 1.042 seconds
```

3. Output / Observed Results

generate_data.py → customers.csv created with 100 synthetic customer records.
database.py → customers.db SQLite database created; Customers table populated.
read_database.py → All 100 records retrieved and printed via SQL SELECT.
api_data.py → 10 user JSON objects returned from JSONPlaceholder API.
profile.py → Column dtypes confirmed; 0 null values detected across all fields.
validate_email.py → All generated email addresses passed regex validation.
duplicate.py → No duplicate records found (unique IDs ensure distinct rows).
clean_data.py → 100 clean records exported to clean_customers.csv.

4. Key Questions / Analysis

● Question / Key Point 1: Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs.

Answer : Schema drift — where an upstream API changes its field names, data types, or nesting structure without notice — is one of the most disruptive failure modes in a data pipeline. Three architectural strategies defend against it. First, schema registry enforcement: every payload is validated against a versioned schema (using tools such as Apache Avro or JSON Schema) before it is allowed to land in the raw zone. When a mismatch is detected, the record is quarantined to a dead-letter queue rather than written to production storage. Second, schema-on-read storage: landing raw payloads in an immutable format such as JSON or Parquet in an object store (S3, GCS) means that structural changes do not corrupt previously ingested data — the schema is only interpreted at query time, allowing retroactive re-parsing when the consuming transformation logic is updated. Third, contract-based API design: producers and consumers agree on a published schema contract and versioning policy (for example, MAJOR version bumps for breaking changes, MINOR for additions). Combined with integration tests that run against a live sandbox environment, these strategies ensure that schema changes are detected during the CI/CD cycle, not during a production incident.

● Question / Key Point 2: What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records?

Answer : Nested JSON offers flexibility and co-location of related data: a single API response can carry an order, its line items, the shipping address, and payment metadata in one atomic document, eliminating the multi-table JOINs that a normalized SQL schema would require to reassemble the same entity. This co-location improves read performance for document-oriented access patterns and makes schema evolution easier — adding a new nested field does not require an ALTER TABLE migration across a large production database. However, the trade-offs are significant. JSON storage is verbose: repeated field names inflate disk usage, and columnar compression techniques work far less efficiently on deeply nested structures compared with flat tabular data stored in Parquet. Querying nested fields is computationally expensive in SQL-over-JSON engines, analytical aggregations (SUM, GROUP BY) are slower than on a properly indexed relational table, and enforcing referential integrity or uniqueness constraints is architecturally impossible without an external validation layer. In practice, the correct choice depends on the access pattern: JSON suits ingestion landing zones and event streaming; normalized SQL suits high-frequency transactional writes and complex relational analytics.

● Question / Key Point 3: How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks?

Answer : The key principle is to decouple validation from transformation. At the ingestion boundary, only lightweight structural checks should execute synchronously — confirming that required primary keys are present, that data types are parseable, and that the payload schema version is recognized. These checks operate at the message level and add sub-millisecond latency per record. Deeper semantic validations (referential integrity checks, cross-field business rule evaluation, statistical outlier detection) are dispatched asynchronously to a parallel validation worker pool, allowing the main ingestion pipeline to continue accepting new records without stalling. Records that fail synchronous checks are immediately routed to a

quarantine store with a structured error envelope containing the original payload, the violation code, and the ingestion timestamp. Records that pass synchronous checks are written to the raw landing zone and flagged for asynchronous re-validation. Monitoring dashboards track the quarantine rate and alert operations teams when the proportion of rejected records exceeds a configured threshold. This approach — synchronous structural gating plus asynchronous semantic validation — provides data quality assurance without introducing latency spikes or throughput degradation in the ingestion tier.

6. Learning Outcomes

After completing this practical, the following skills were developed:

1. Programmatic synthetic data generation using the Faker library.
2. Reading and writing CSV files using Pandas.
3. Creating and querying SQLite databases from Python.
4. Integrating external REST APIs using the Requests library.
5. Profiling dataset structure, data types, and null-value distributions.
6. Applying regular-expression-based email format validation.
7. Detecting and reporting duplicate records in a dataset.
8. Removing null-containing rows and exporting a clean dataset.
9. Designing an automated file-scanning and quarantine routine.
10. Writing structured warning entries to application log files.

8. Conclusion

This practical demonstrated the complete data source profiling and ingestion preparation workflow. By generating synthetic data, storing it in a relational database, fetching external API data, profiling the dataset, validating field formats, detecting duplicates, and cleaning null entries, the core data engineering skills required before loading data into a production pipeline were exercised. The supplementary quarantine routine extended this into a realistic automated quality gate, and the post-laboratory log export reflects industry-standard observability practices applied to incoming data payloads.